TEACHING WALLAH

Teaching Exam

Subject – Computer Science

Data Structure and Algorithm

Lecture No -4

By-Satyendra Chaudhary Sir

Topics to be Covered

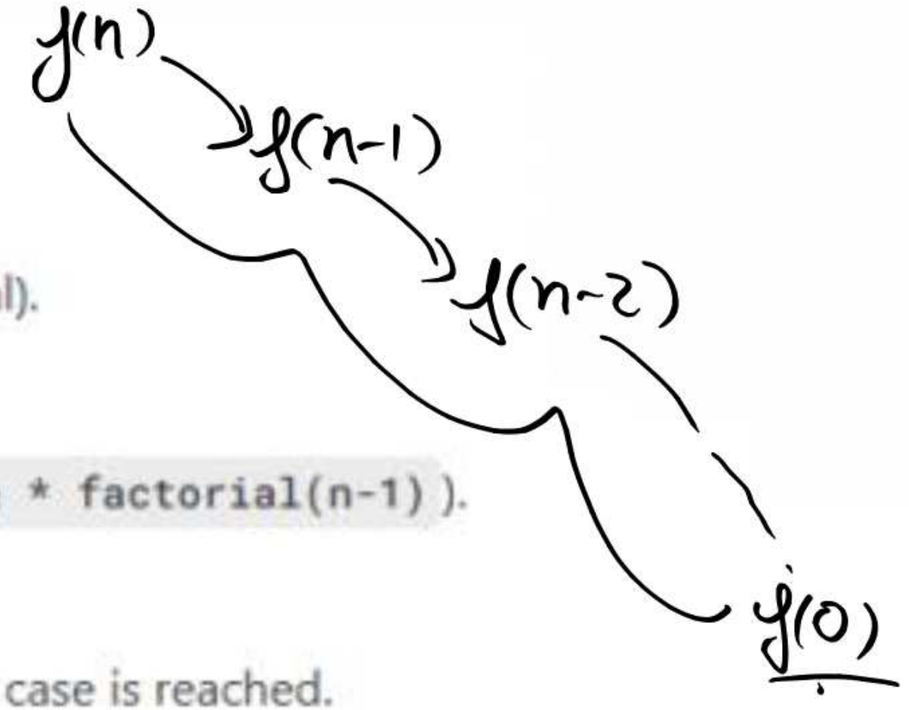
Topic

Queue.

Linked List.

Tree.

Recursion is a programming technique where a function calls itself to solve a problem by breaking it down into smaller, similar subproblems. Here are two easy-to-understand examples with explanations and flowcharts.



Key Concepts of Recursion:

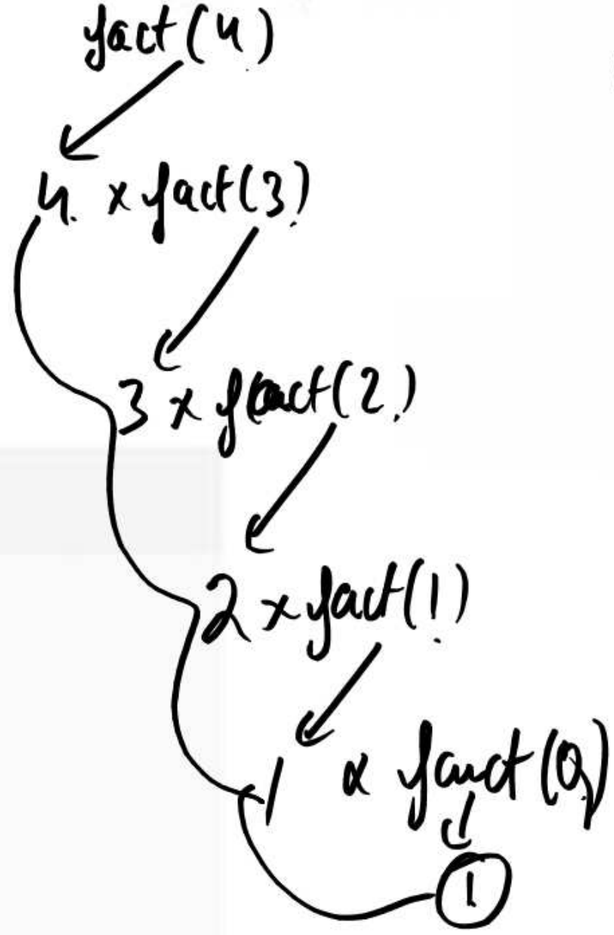
1. **Base Case:**
   - A condition to stop recursion (e.g., `n == 0` in factorial).
2. **Recursive Case:**
   - The function calls itself with a smaller problem (e.g., `n * factorial(n-1)`).
3. **Call Stack:**
   - Each recursive call is added to the stack until the base case is reached.
4. **Unwinding:**
   - The stack resolves backwards once the base case is met.

Example 1: Factorial Calculation

The factorial of a number `n` (denoted as `n!`) is the product of all positive integers up to `n`.

Recursive Definition:

- `n! = n × (n-1)!`
- Base Case: `0! = 1`



4x3x2x1=24

Python Code:

```python
def factorial(n):
    if n == 0:  # Base case
        return 1
    else:
        return n * factorial(n - 1)  # Recursive call

print(factorial(3))  # Output: 6 (3! = 3 × 2 × 1)
```

Execution Flow (for `factorial(3)`):

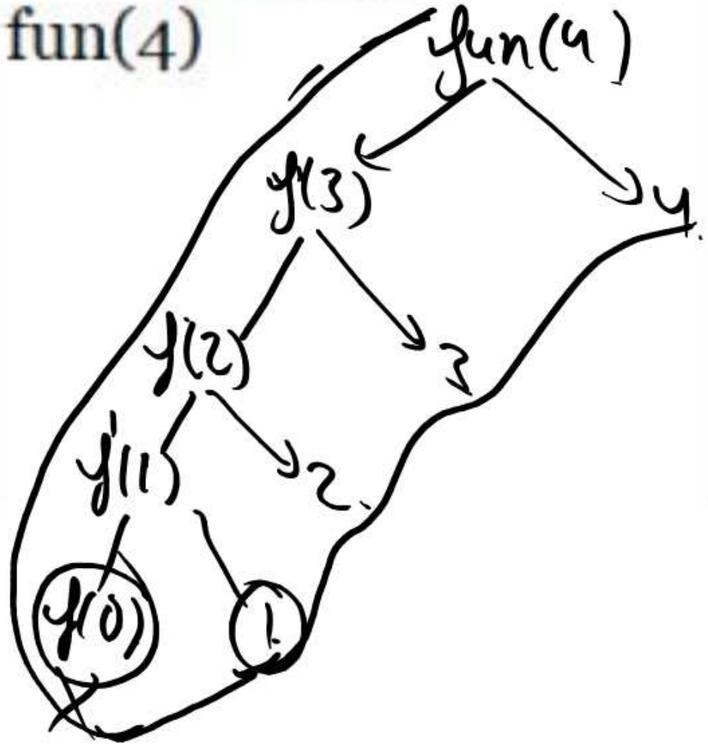
1. Call `factorial(3)`
   - `3 != 0` → `return 3 × factorial(2)`
2. Call `factorial(2)`
   - `2 != 0` → `return 2 × factorial(1)`
3. Call `factorial(1)`
   - `1 != 0` → `return 1 × factorial(0)`
4. Call `factorial(0)`
   - `0 == 0` → `return 1` (Base case stops recursion).
5. Unwinding the Calls:
   - `factorial(1) = 1 × 1 = 1`
   - `factorial(2) = 2 × 1 = 2`
   - `factorial(3) = 3 × 2 = 6` → Final result.

```
factorial(3)
   ↓
3 * factorial(2)
       ↓
    2 * factorial(1)
           ↓
        1 * factorial(0)
               ↓
             1 (Base Case)
```

hello.py - C:\Users\TESTBOOK\Desktop\hell

File Edit Format Run Options Window

```
def fun(x):
    if(x>0):
        fun(x-1)
        print (x)
fun(4)
```



IDLE Shell 3.13.3

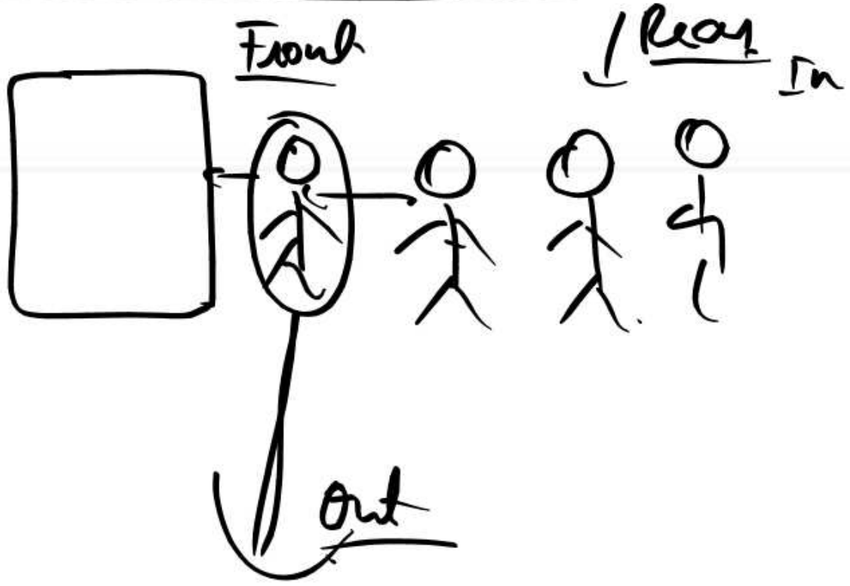
File Edit Shell Debug Options Window Help

```
Python 3.13.3 (tags/v3.13.3:
Enter "help" below or click
>>>

========== R
1
2
3
4
>>>
```

What is a Queue?

- A **linear data structure** that follows the **FIFO (First In First Out)** principle
- Elements are inserted at the **rear** and removed from the **front**

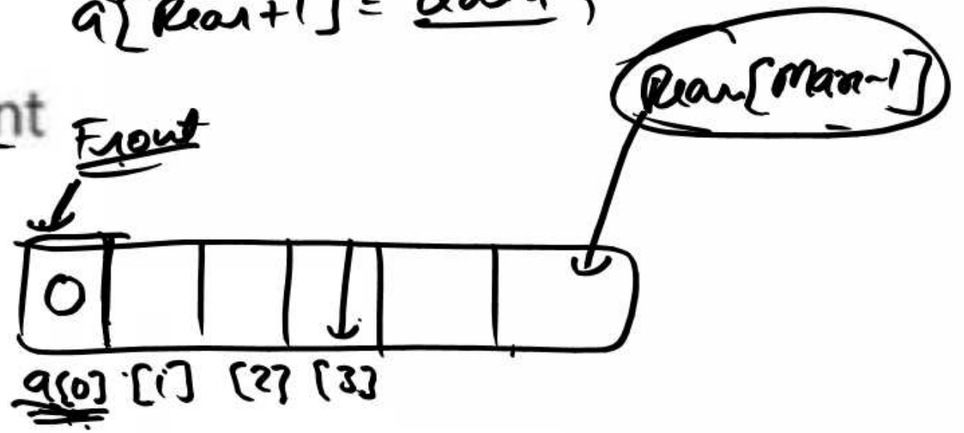


: Queue Terminologies

- **Enqueue**: Inserting an element at the rear
- **Dequeue**: Removing an element from the front
- **Front**: Index of the first element
- **Rear**: Index of the last element
- **Overflow**: Insertion when the queue is full
- **Underflow**: Deletion when the queue is empty

Insertion

```
Rear = Rear+1 ;
q[Rear+1] = data ;
```



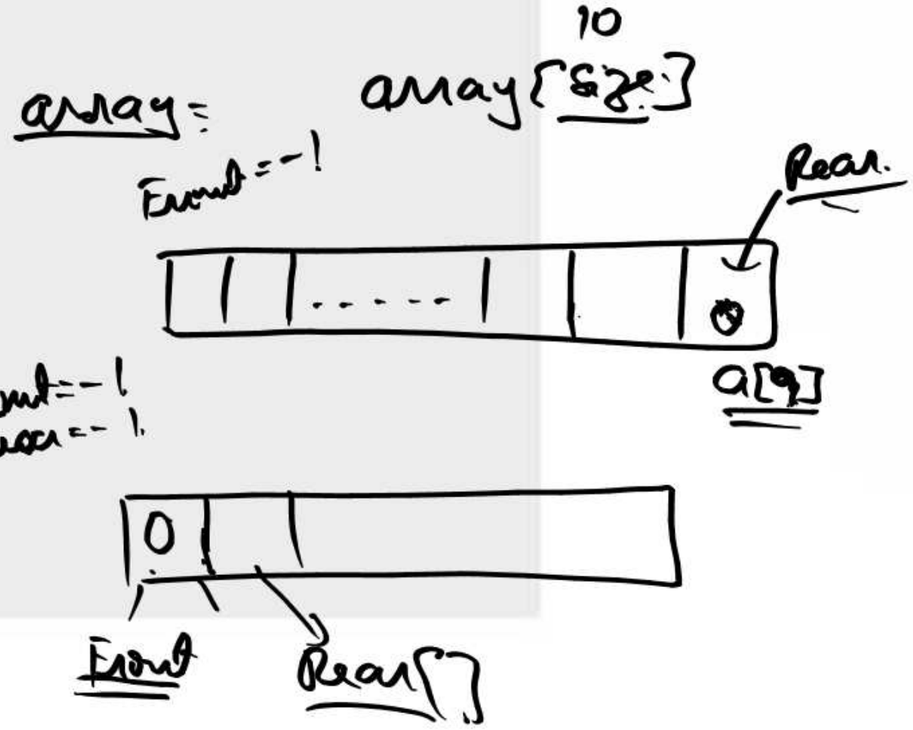
Front = -1
Rear = -1

Front = 0
Rear = 0

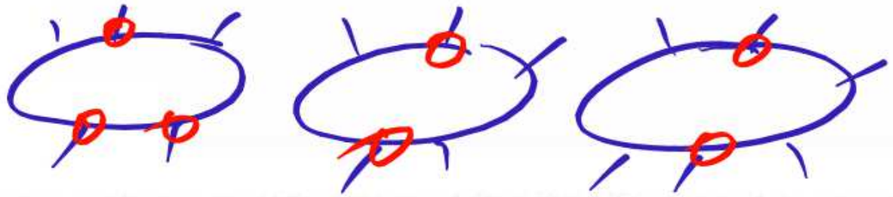
Front = 0
Rear = 1

Pseudocode for Insertion (Enqueue)

```
Procedure ENQUEUE(queue, item):
    if rear == size - 1:
        print "Overflow"
    else:
        if front == -1:
            front = 0
        rear = rear + 1
        queue[rear] = item
```



F=0 F=0
R=0 R=(1)



Linear Queue

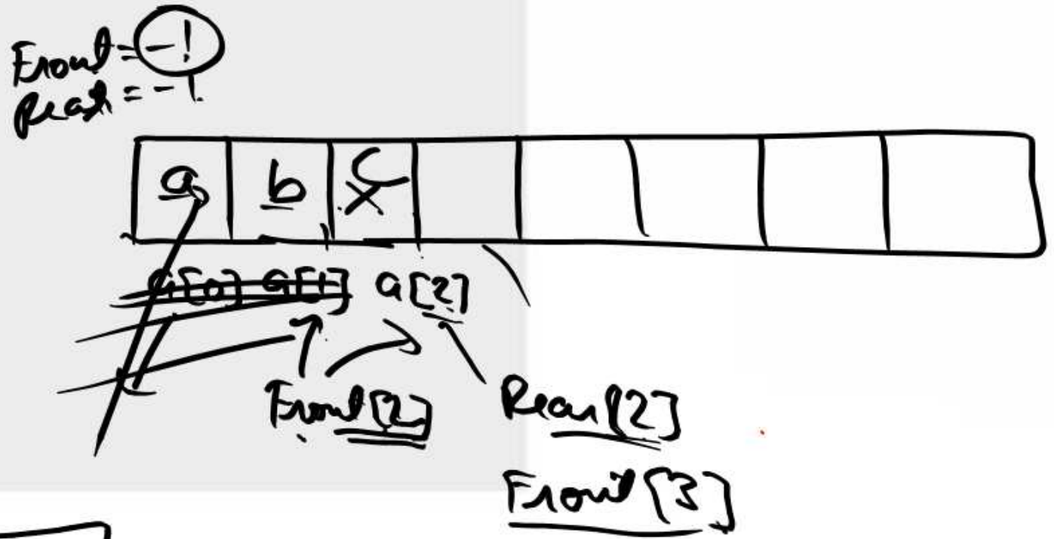
TEACHING WALLAH

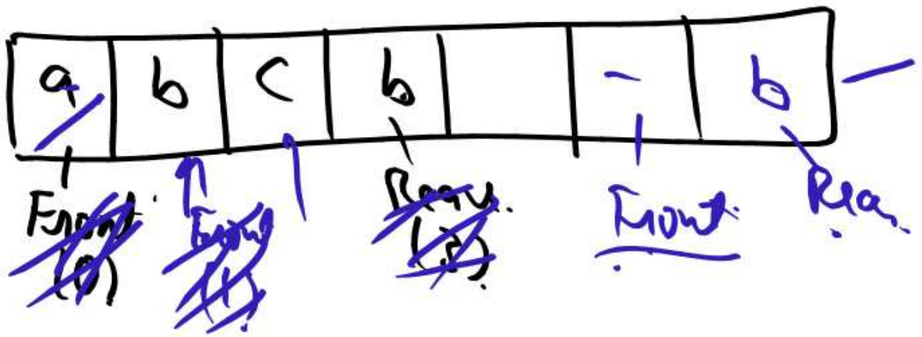
Pseudocode for Deletion (Dequeue)

```
Procedure DEQUEUE(queue):
    if front == -1 or front > rear:
        print "Underflow"
    else:
        item = queue[front]
        front = front + 1
        return item
```

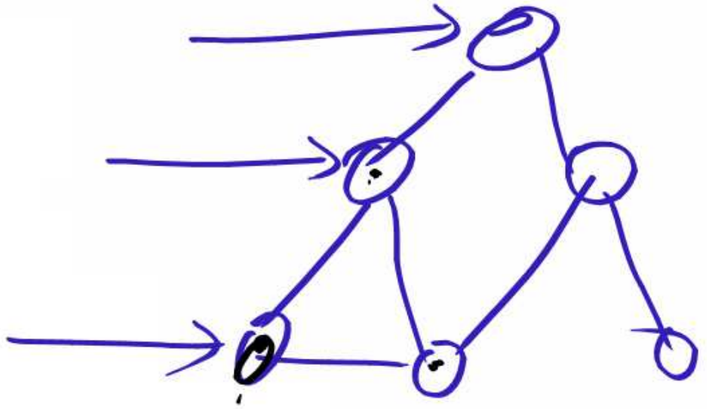
size

array[8]





Uses of Queue



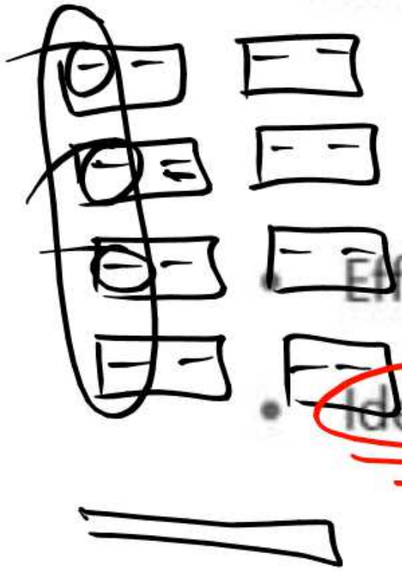
- CPU and Disk Scheduling
- Printer Spooling
- Handling requests in web servers
- Breadth-First Search (BFS) in Graphs

: Disadvantages of Linear Queue

- Fixed size (cannot grow dynamically)
- Inefficient space usage due to unused front space

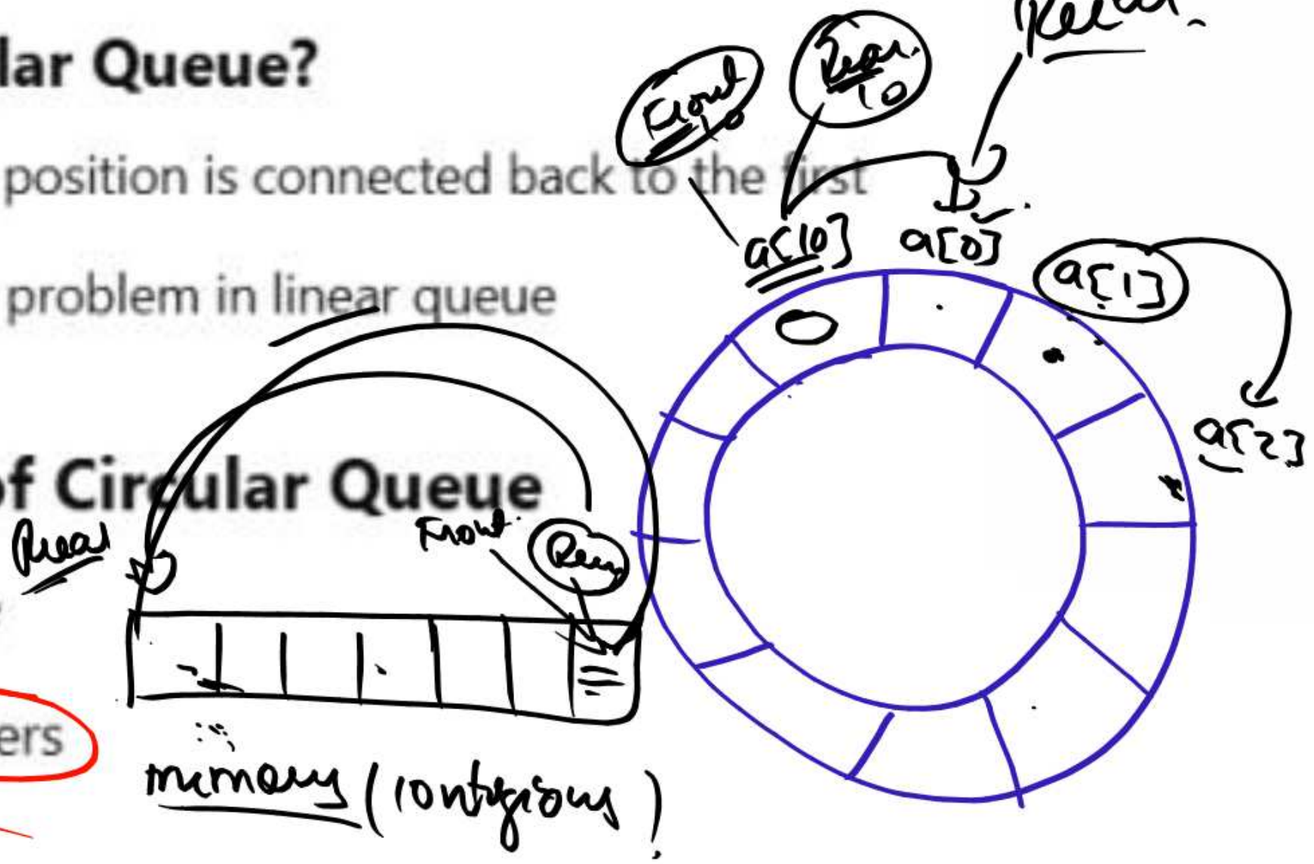
What is a Circular Queue?

- A queue in which the last position is connected back to the first
- Solves the space wastage problem in linear queue



: Advantages of Circular Queue

- Efficient memory usage
- Ideal for fixed-size buffers



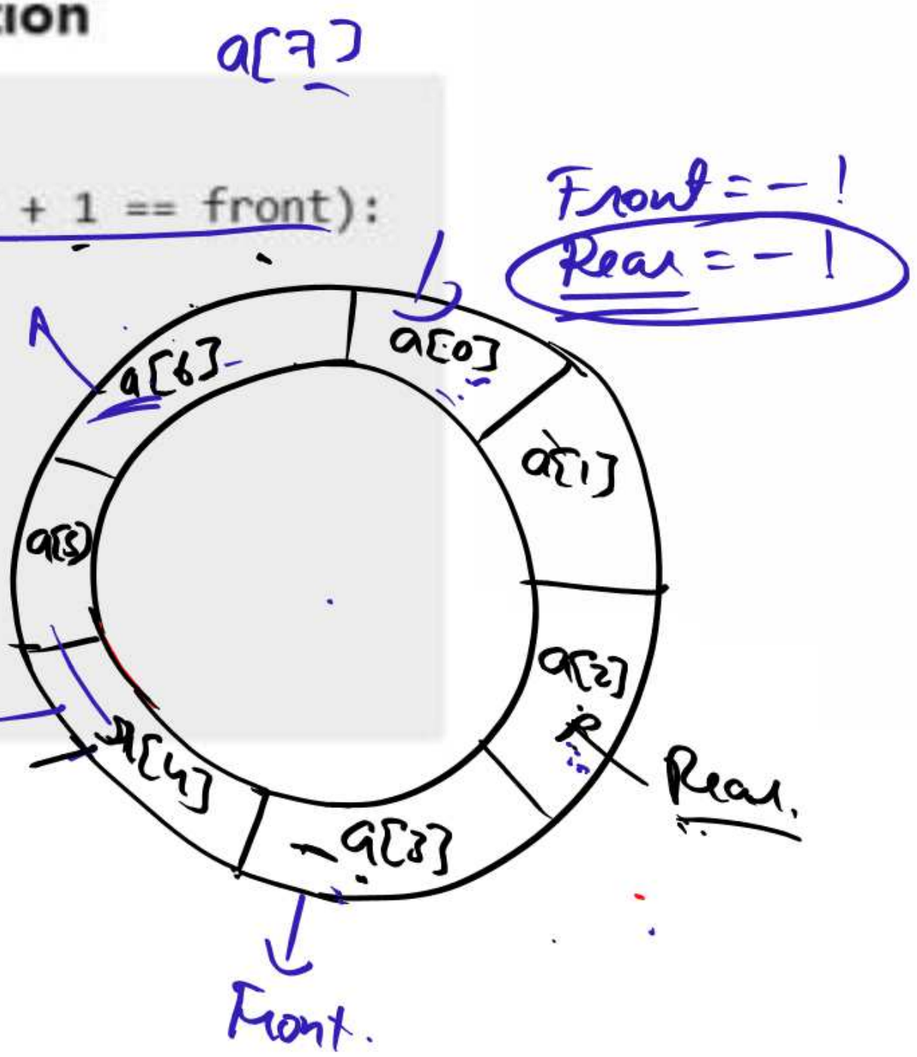
Pseudocode for Circular Queue Insertion

```
Procedure CIRCULAR_ENQUEUE(queue, item):
    if (front == 0 and rear == size-1) or (rear + 1 == front):
        print "Overflow"
    else:
        if front == -1:
            front = rear = 0
        else:
            rear = (rear + 1) % size
        queue[rear] = item
```

Renamed

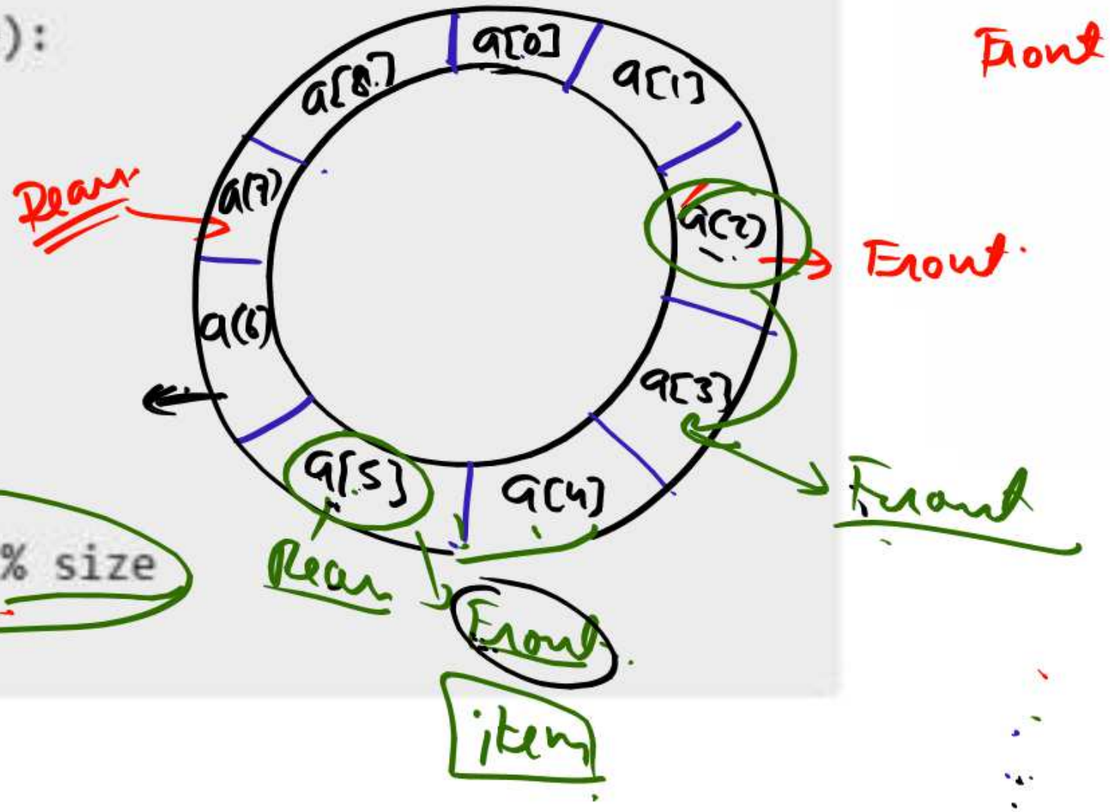
5
7
Rear(5)

rear = (4+1) % 7.



Pseudocode for Circular Queue Deletion

```
Procedure CIRCULAR_DEQUEUE(queue):
    if front == -1:
        print "Underflow"
    else:
        item = queue[front]
        if front == rear:
            front = rear = -1
        else:
            front = (front + 1) % size
        return item
```



Conclusion

- Queues are essential for ordered processing
- Circular queues solve space limitations of linear queues
- Proper implementation ensures efficient performance in various applications

Time Complexities of Queue Operations

Operation	Linear Queue	Circular Queue	Stack
Enqueue / Push	O(1)	O(1)	O(1)
Dequeue / Pop	O(1)	O(1)	O(1)
Peek / Front	O(1)	O(1)	O(1)
IsEmpty / IsFull	O(1)	O(1)	O(1)

1. A linear list of elements in which deletion can be done from one end (front) and insertion can take place only at the other end (rear) is known as _____

a) Queue
b) Stack
c) Tree
d) Linked list

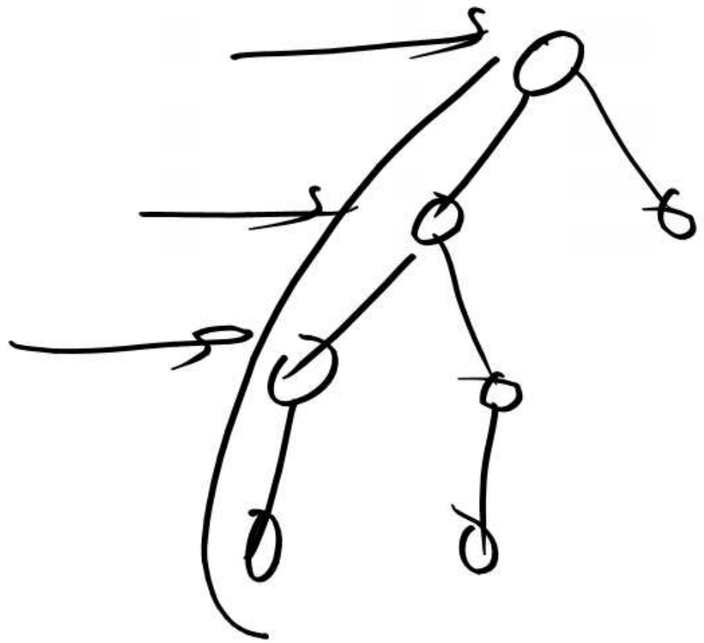
2. The data structure required for Breadth First Traversal on a graph is?

a) Stack

b) Array

c) Queue

d) Tree



3. A queue follows _____

a) FIFO (First In First Out) principle

b) LIFO (Last In First Out) principle

c) Ordered array

d) Linear tree

4. Circular Queue is also known as _____

a) Ring Buffer

b) Square Buffer

c) Rectangle Buffer

d) Curve Buffer

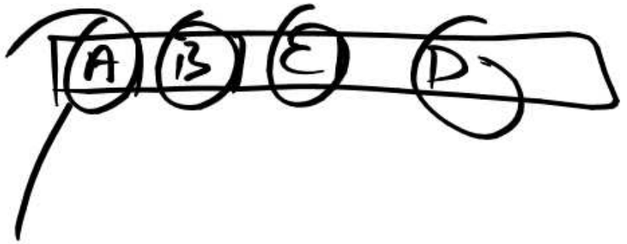
5. If the elements “A”, “B”, “C” and “D” are placed in a queue and are deleted one at a time, in what order will they be removed?

a) ABCD

b) DCBA

c) DCAB

d) ABDC



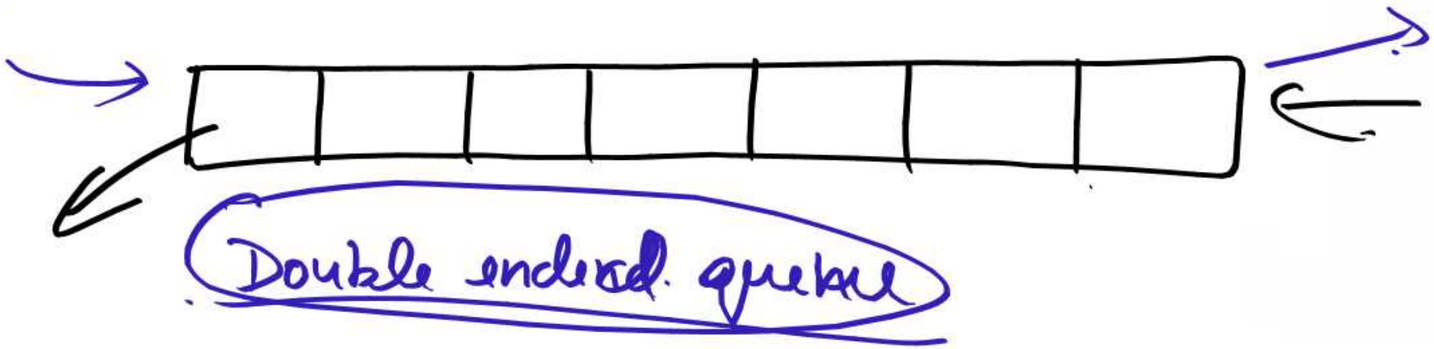
6. A data structure in which elements can be inserted or deleted at/from both ends but not in the middle is?

a) Queue

b) Circular queue

c) Dequeue

d) Priority queue



Double ended queue

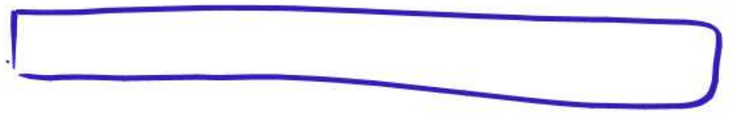
7. A normal queue, if implemented using an array of size MAX_SIZE, gets full when?

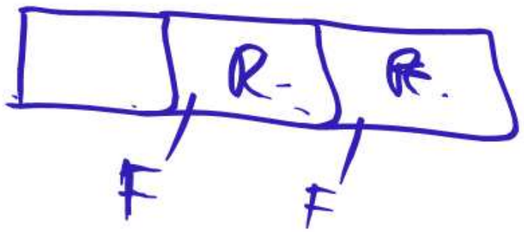
a) Rear = MAX_SIZE – 1

b) Front = (rear + 1)mod MAX_SIZE

c) Front = rear + 1

d) Rear = front





8. Queues serve major role in _____

a) Simulation of recursion x-stack.

b) Simulation of arbitrary linked list

c) Simulation of limited resource allocation

d) Simulation of heap sort

_____

9. Which of the following is not the type of queue?

a) Ordinary queue

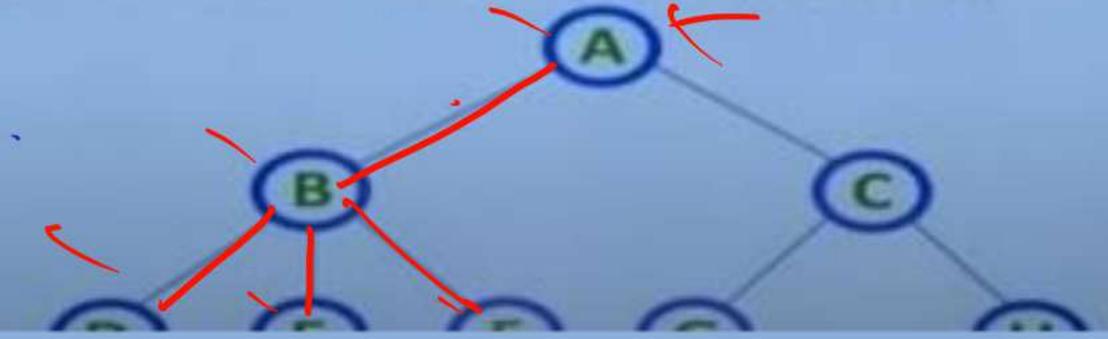
b) Single ended queue

c) Circular queue

d) Priority queue

Data Structure	Access	Insert	Delete	Use Case
Array	O(1)	O(n)	O(n)	Random access, fixed size
Linked List	O(n)	O(1)/O(n)	O(1)/O(n)	Dynamic size, insertion-heavy
Stack	O(1)	O(1)	O(1)	Recursion, parsing
Queue	O(1)	O(1)	O(1)	Task scheduling
Hash Table	O(1)*	O(1)*	O(1)*	Fast lookup by key
Tree (BST)	O(log n)	O(log n)	O(log n)	Sorted data, hierarchy
Heap	O(n)	O(log n)	O(log n)	Priority handling
Graph	Varies	Varies	Varies	Network modeling
Trie	O(L)	O(L)	O(L)	Prefix search

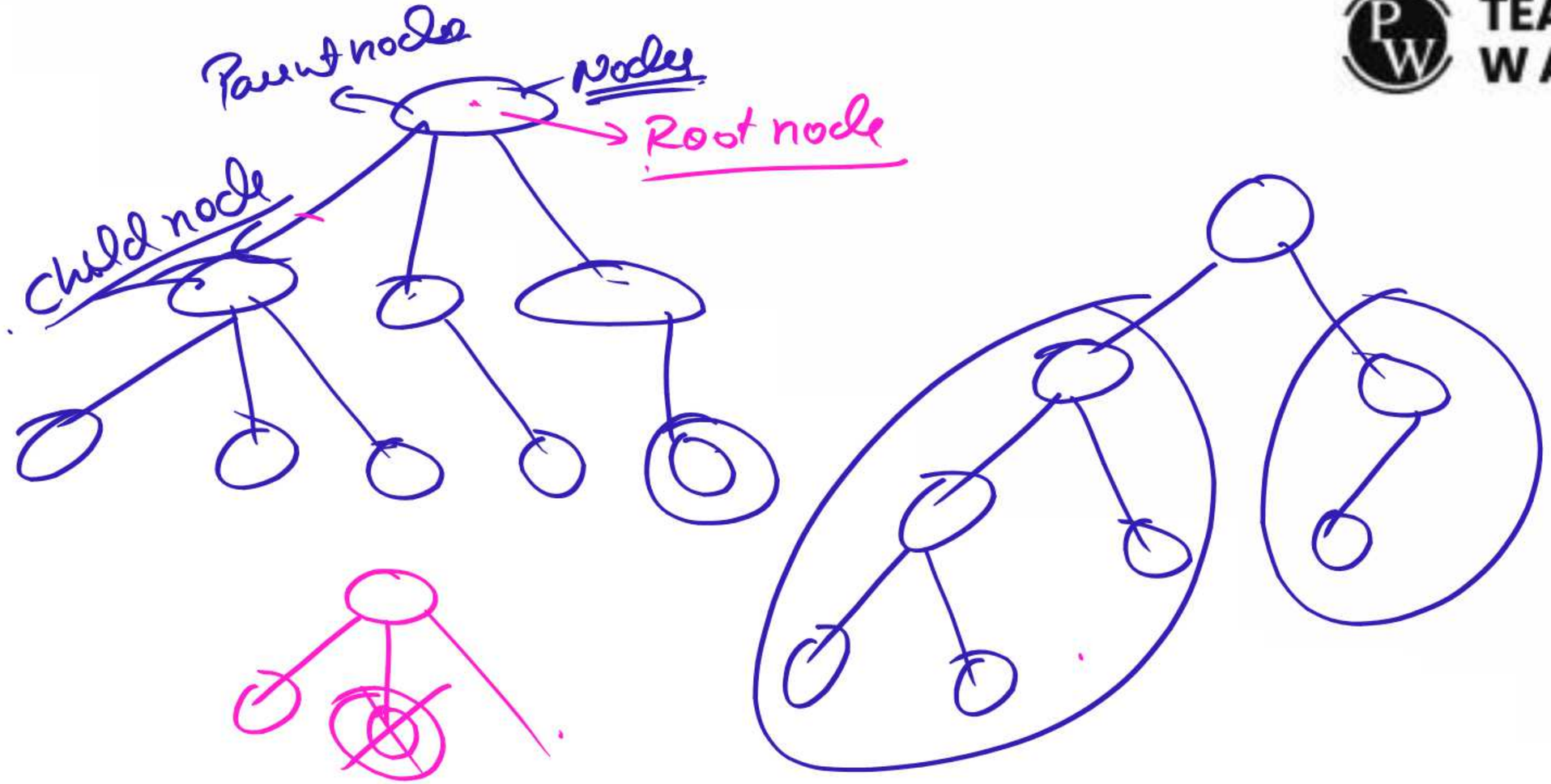
- **Root Node**: The top-most node, serving as the origin of the tree.
- **Edge**: The connection between two nodes. A tree with 'N' nodes has exactly 'N-1' edges.
- **Parent Node**: A node that has one or more children.
- **Child Node**: A node that descends from a parent node.
- **Leaf (External) Node**: A node with no children.
- **Internal Node**: A node with at least one child.
- **Degree of Node**: The number of children a node has.
- **Level/Depth**: Indicates the step count from the root (Level 0) to a particular node.
- **Path**: A sequence of nodes connected by edges.
- **Subtree**: A tree formed from a child node and its descendants.

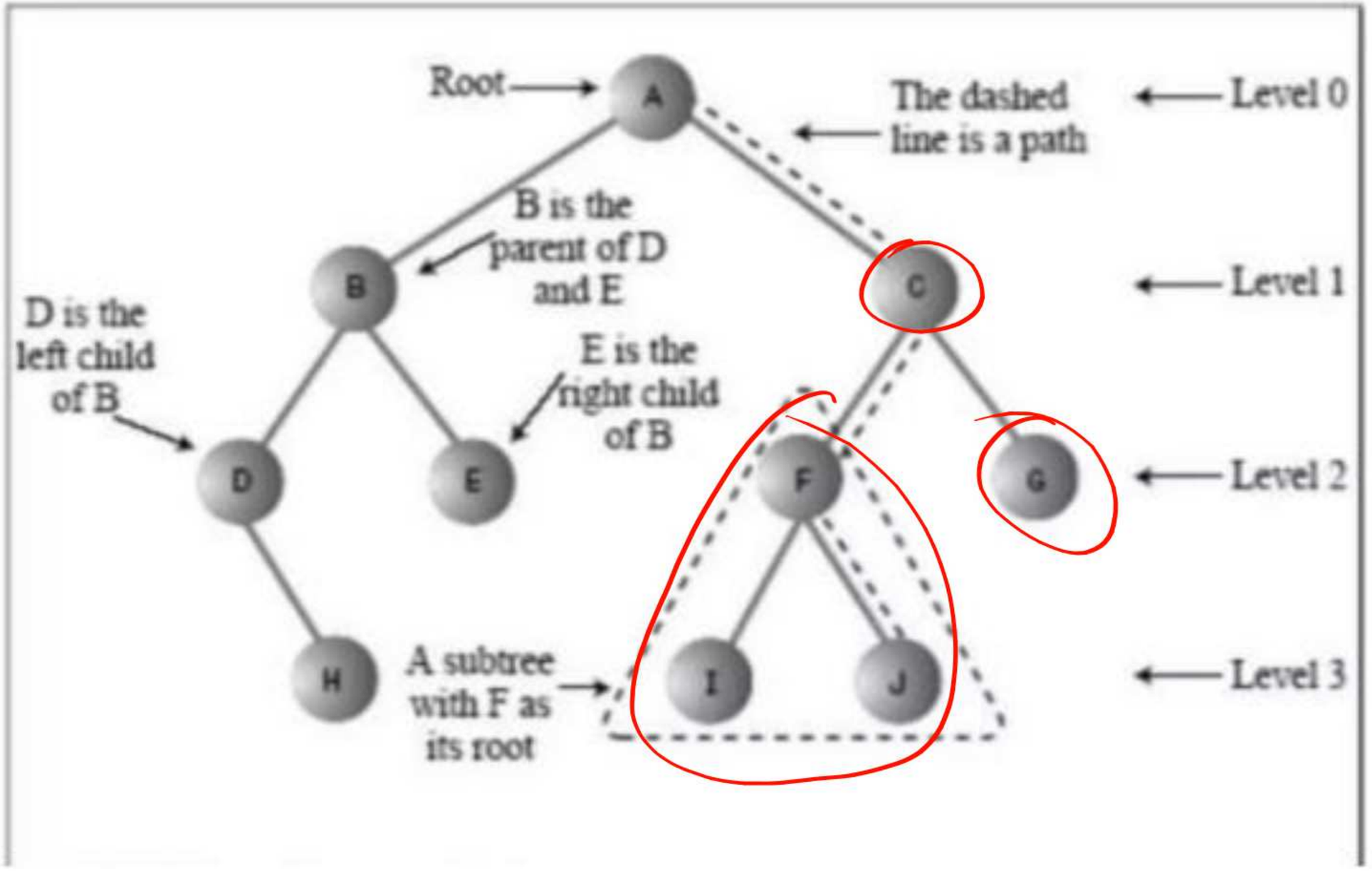


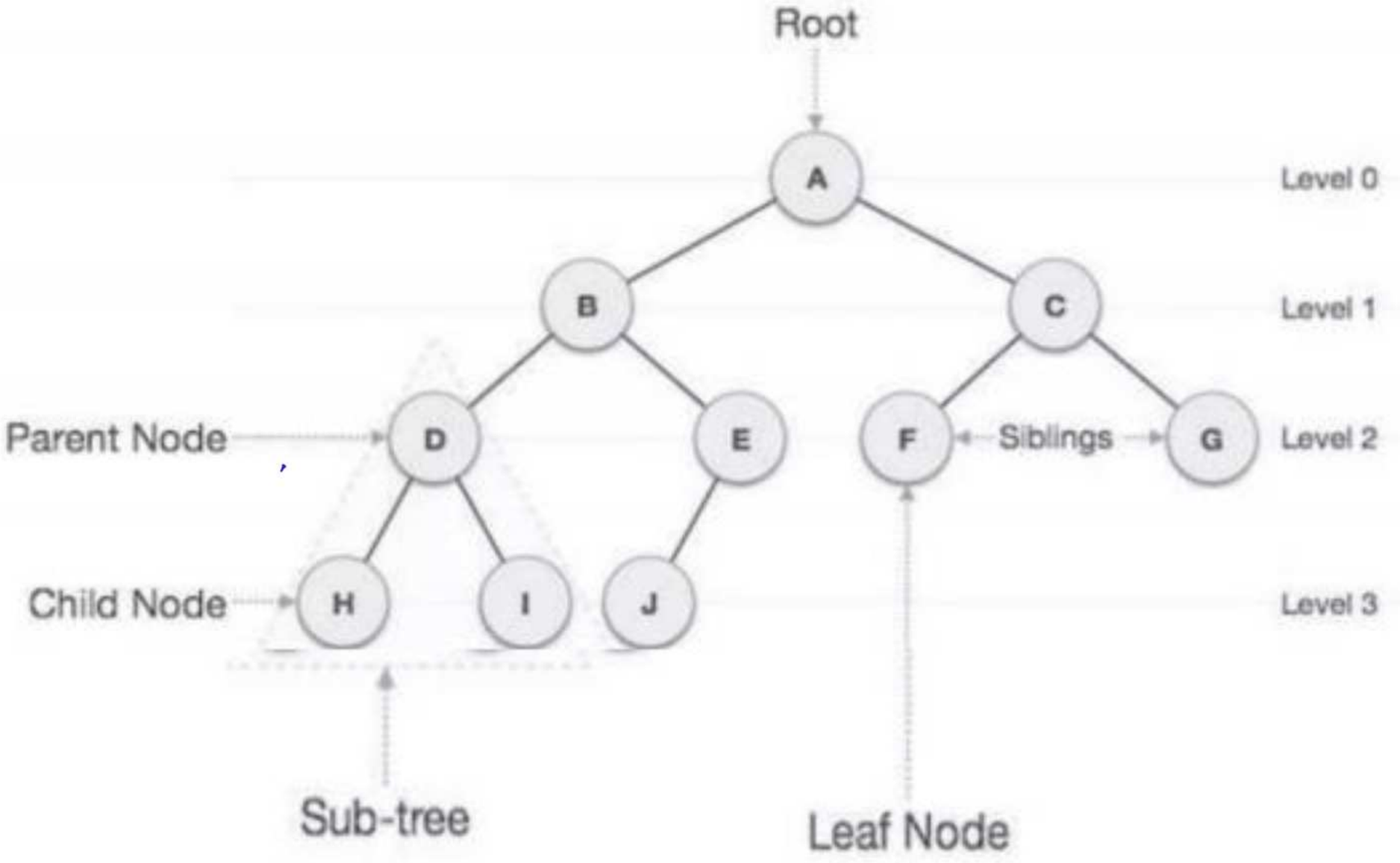
Binary tree

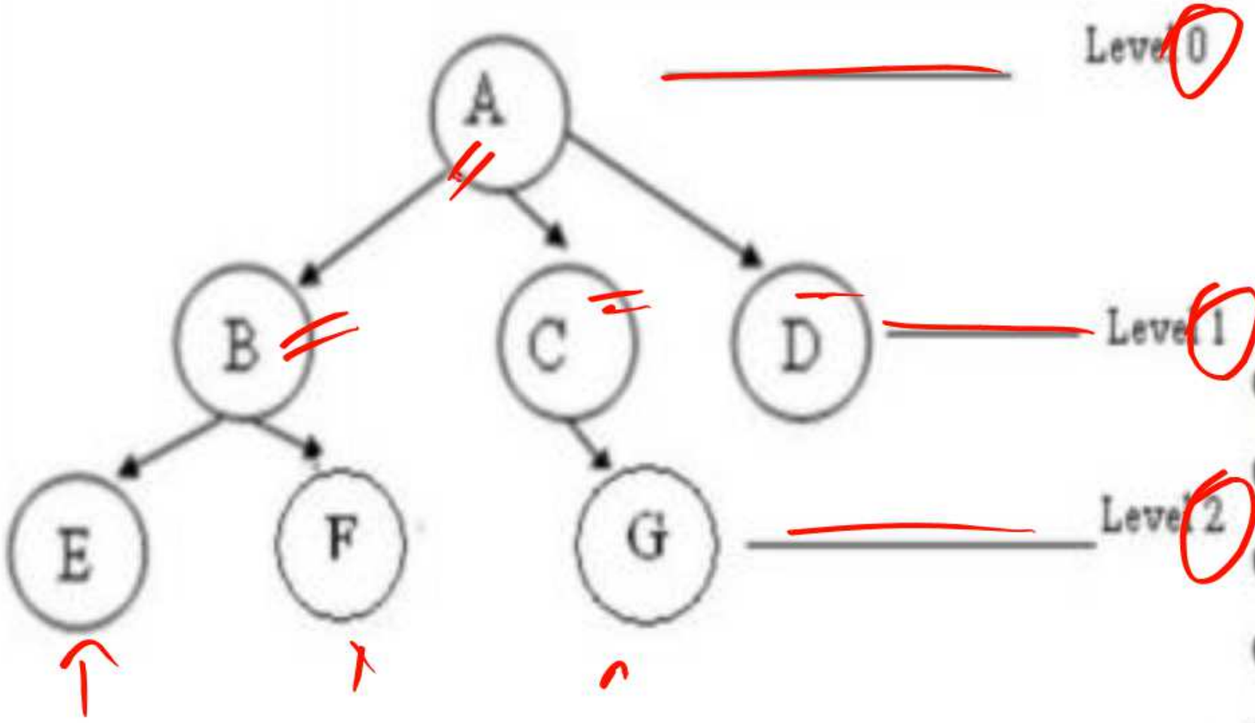
- A **binary tree** is a type of tree data structure where each node can have at most **two children**, commonly referred to as **left** and **right** child nodes. It is structured as a set of nodes with the following properties:
- **Empty Tree:** A binary tree can be empty (no nodes).
- **Root Node:** If not empty, the tree has a unique root node (R).
- **Subtrees:** The remaining nodes are partitioned into two disjoint subtrees, **T1** (left) and **T2** (right).
- Representation of tree in memory
  - Sequential representation – using an array info and left child and right child
  - Linked Representation – using self-referential structure node


```
struct node {
   int data;
   struct node* left;
   struct node* right;
}
```







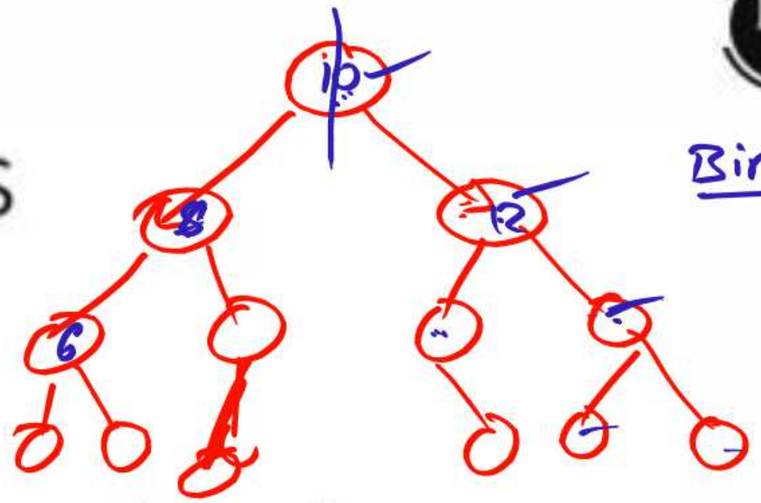


- ✔ A is the root node
- ✔ B is the parent of E and F
- ✔ D is the sibling of B and C
- ✔ E and F are children of B
- ✔ E, F, G, D are external nodes or leaves
- ✔ A, B, C are internal nodes
- ✔ Depth of F is 2
- ✔ the height of tree is 2
- ✔ the degree of node A is 3
- ✔ The degree of tree is 3

a[10]

10

Characteristics of trees



Binary Search

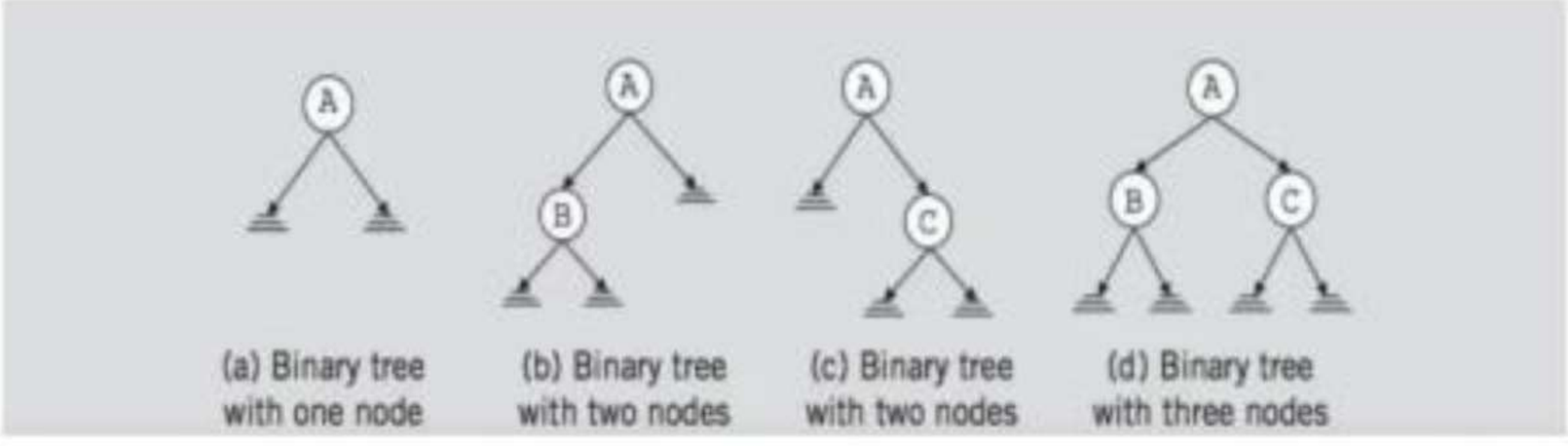
- Non-linear data structure
- Combines advantages of an ordered array
- Searching as fast as in ordered array
- Insertion and deletion as fast as in linked list
- Simple and fast

Application

- Directory structure of a file store
- Structure of an arithmetic expressions
- Used in almost every 3D video game to determine what objects need to be rendered.
- Used in almost every high-bandwidth router for storing router-tables.
- used in compression algorithms, such as those used by the .jpeg and .mp3 file-formats.

Binary Trees

- A **binary tree**, *T*, is either empty or such that
  - *I.* *T* has a special node called the **root** node
  - *II.* *T* has two sets of nodes L_T and R_T, called the **left subtree** and **right subtree** of *T*, respectively.
  - *III.* L_T and R_T are binary trees.

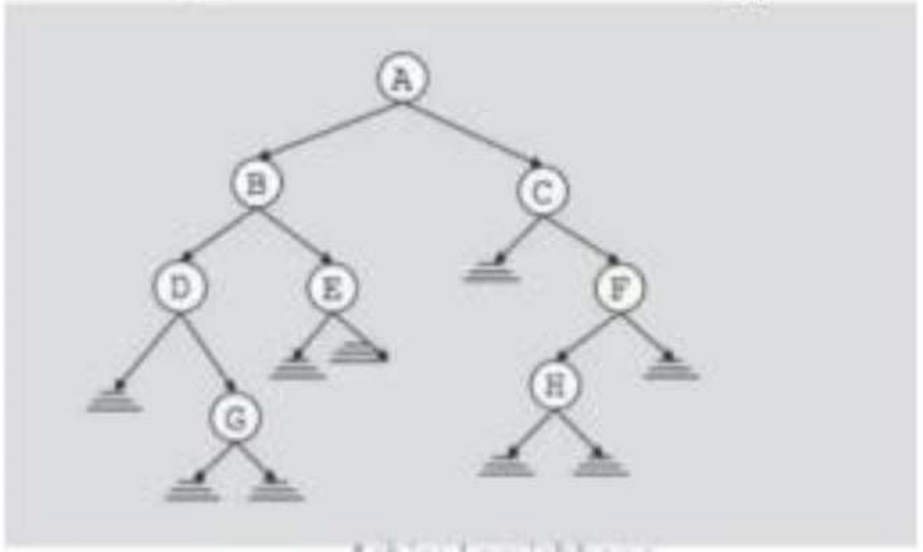


Binary Tree

- The root node of this binary tree is A.

- The left sub tree of the root node, which we denoted by L_A, is the set L_A = {B,D,E,G} and the right sub tree of the root node, R_A is the set R_A={C,F,H}

- The root node of L_A is node B, the root node of R_A is C and so on



Binary Tree Properties

- If a binary tree contains **m** nodes at level **L**, it contains at most **2m** nodes at level **L+1**

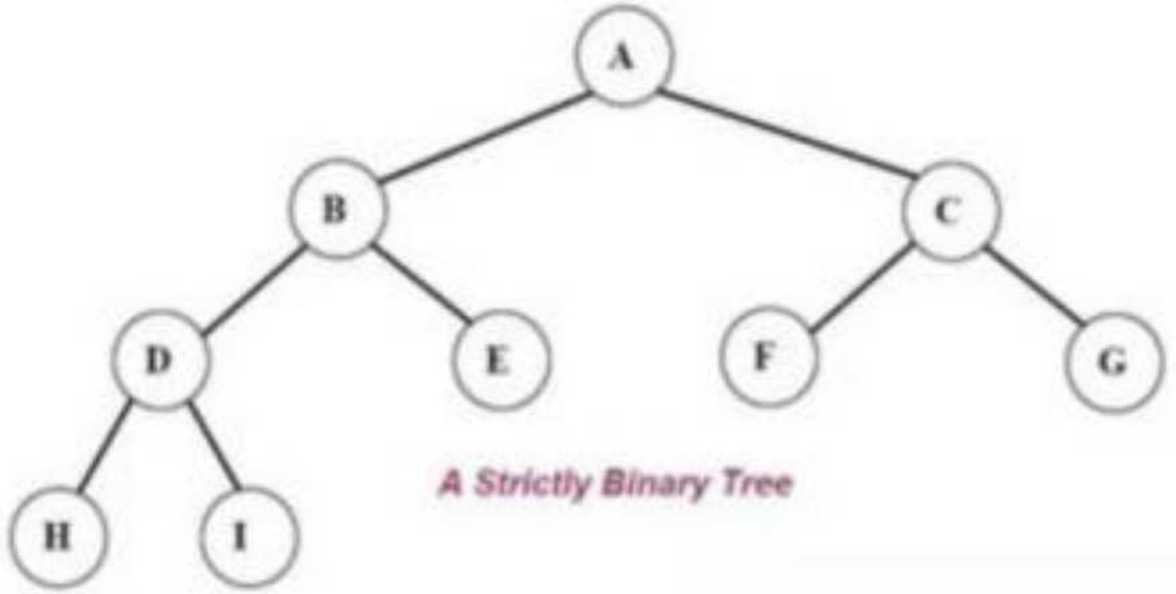
- Since a binary tree can contain at most 1 node at level 0 (the root), it contains at most **2L** nodes at level L.

Types of Binary Tree

- Complete binary tree
- Strictly binary tree
- Almost complete binary tree

- **Or, to put it another way,** all of the nodes in a strictly binary tree are of degree zero or two, never degree one.

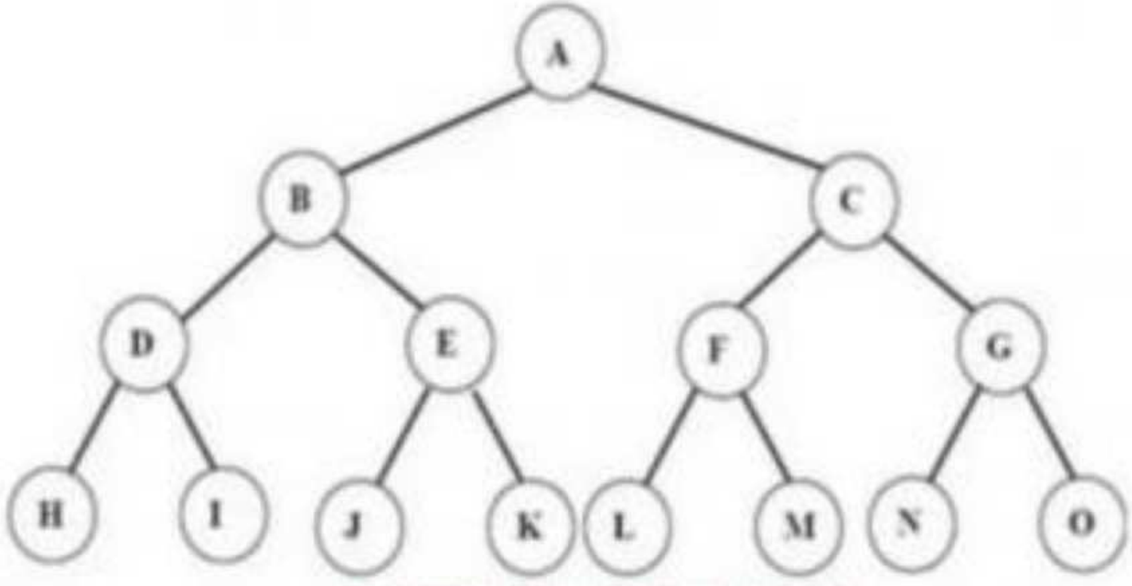
- A strictly binary tree with N leaves always contains 2N – 1 nodes.



A Strictly Binary Tree

- A ***complete binary tree*** of depth **d** is called strictly binary tree if all of whose leaves are at level **d**.

- A complete binary tree has 2^d nodes at every depth **d** and 2^d -1 non leaf nodes



A complete Binary Tree of depth 3

- An almost complete binary tree is a tree where for a right child, there is always a left child, but for a left child there may not be a right child.

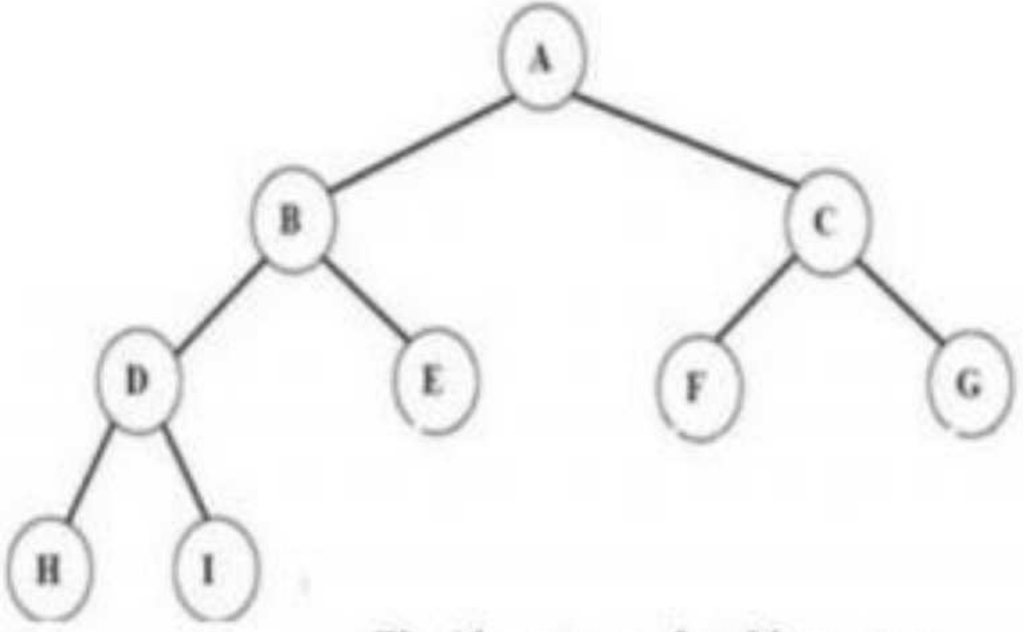


Fig Almost complete binary tree.

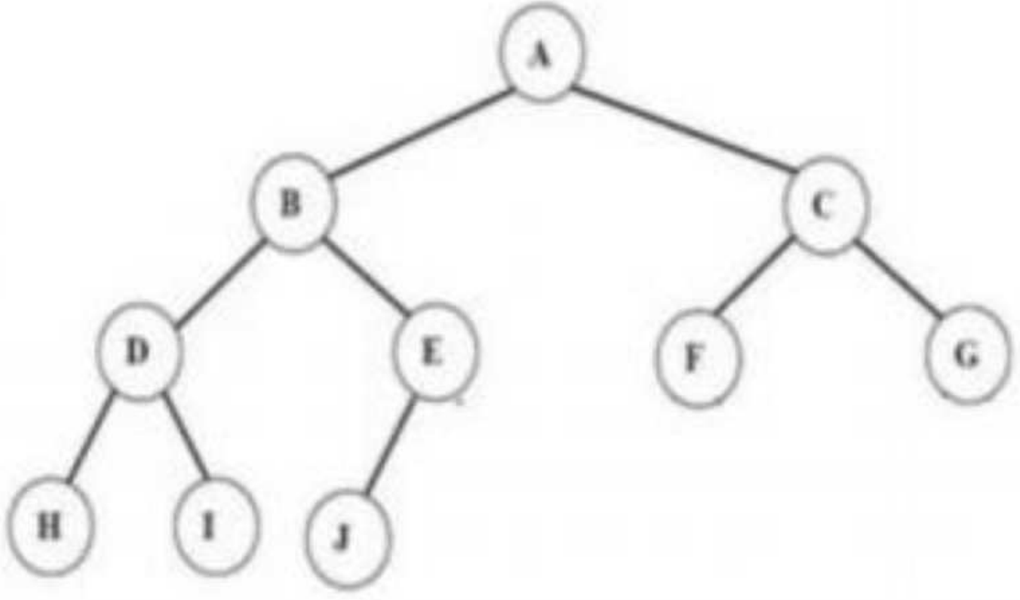


Fig Almost complete binary tree but not strictly
Since node E has a left son but not a right son.

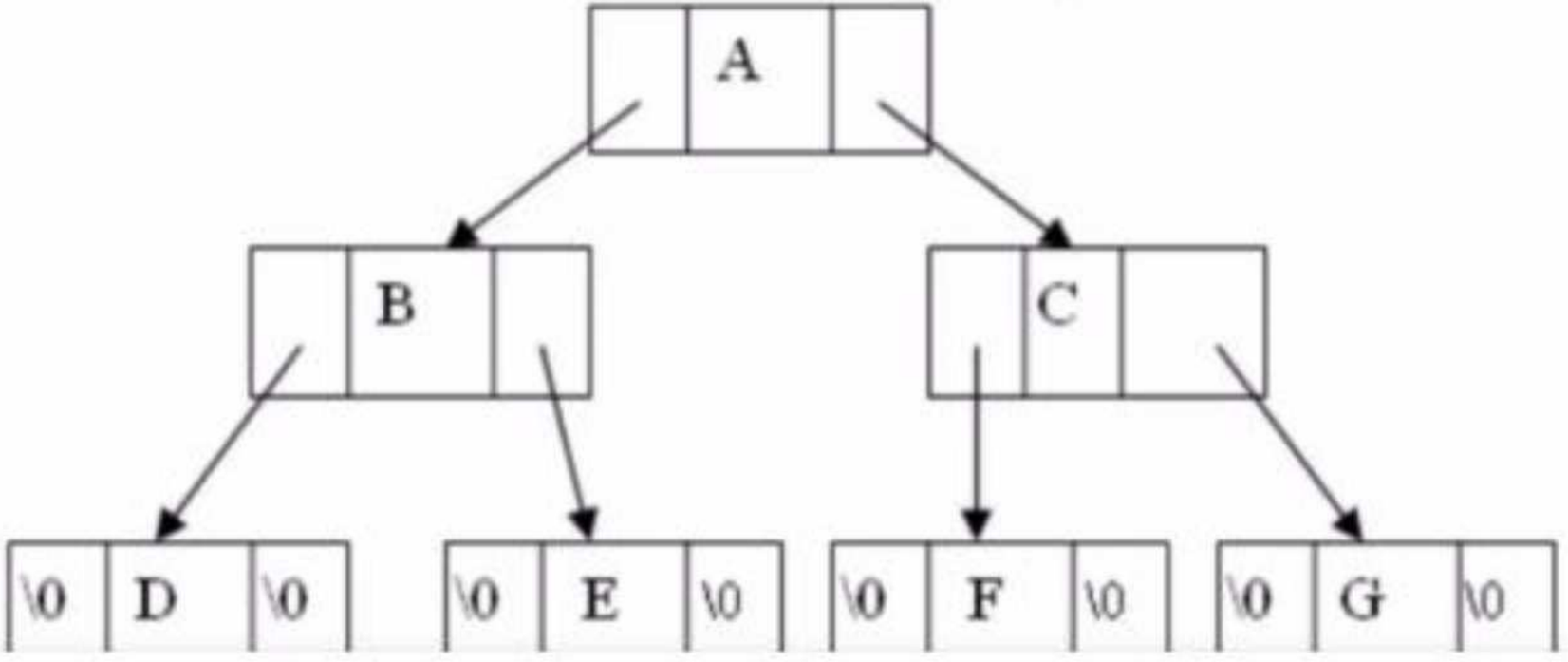
Operations on Binary tree:

- ✔ **father(n,T):**Return the parent node of the node n in tree T. If n is the root, NULL is returned.
- ✔ **LeftChild(n,T):**Return the left child of node n in tree T. Return NULL if n does not have a left child.
- ✔ **RightChild(n,T)**:Return the right child of node n in tree T. Return NULL if n does not have a right child.
- ✔ **Info(n,T):** Return information stored in node n of tree T (ie. Content of a node).
- ✔ **Sibling(n,T):** return the sibling node of node **n** in tree T. Return NULL if n has no sibling.

- ✔ **Root(T):** Return root node of a tree if and only if the tree is nonempty.
- ✔ **Size(T):** Return the number of nodes in tree T
- ✔ **MakeEmpty(T):** Create an empty tree T
- ✔ **SetLeft(S,T):** Attach the tree S as the left sub-tree of tree T
- ✔ **SetRight(S,T):** Attach the tree S as the right sub-tree of tree T.
- ✔ **Preorder(T):** Traverses all the nodes of tree T in preorder.
- ✔ **postorder(T):** Traverses all the nodes of tree T in postorder
- ✔ **Inorder(T):** Traverses all the nodes of tree T in inorder.

C representation for Binary tree:

```
struct bnode
{
        int info;
        struct bnode  *left;
        struct bnode  *right;
};
struct bnode *root=NUI
```



Tree traversal

- Traversal is a process to visit all the nodes of a tree and may print their values too.

- All nodes are connected via edges (links) we always start from the root (head) node.

- There are three ways which we use to traverse a tree
  - In-order Traversal
  - Pre-order Traversal
  - Post-order Traversal

- Pre-order

<root><left><right>

- In-order

<left><root><right>

- Post-order

<left><right><root>

- The preorder traversal of a nonempty binary tree is defined as follows:
  - Visit the root node
  - Traverse the left sub-tree in preorder
  - Traverse the right sub-tree in preorder

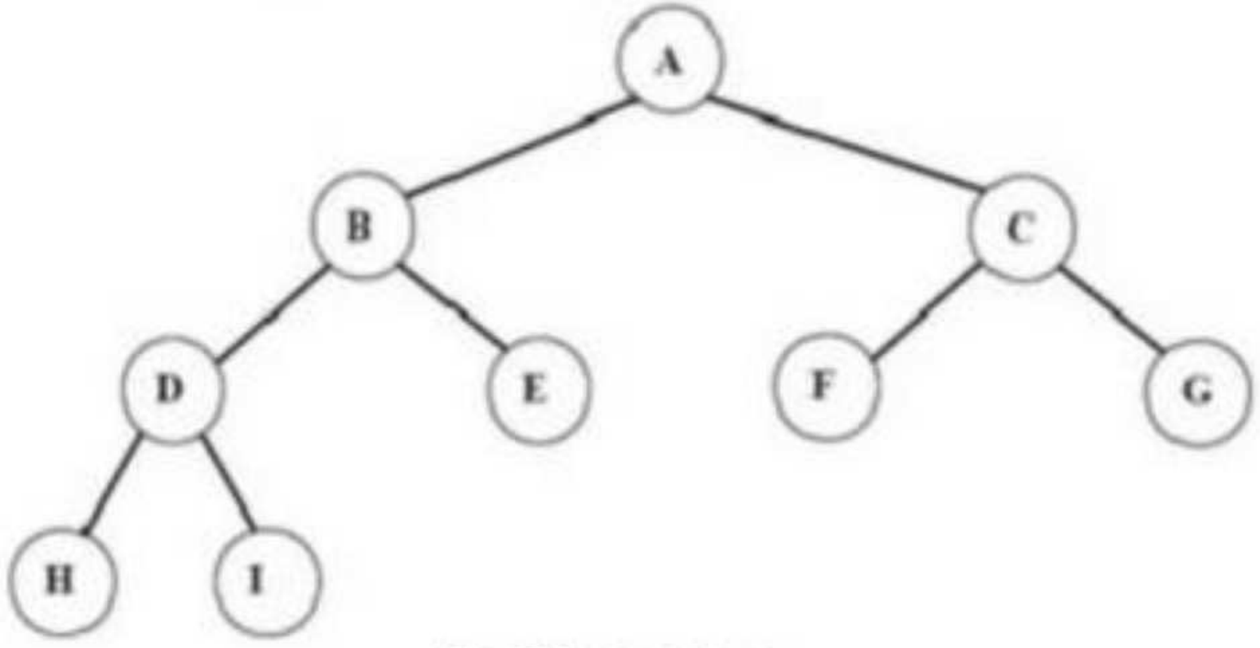
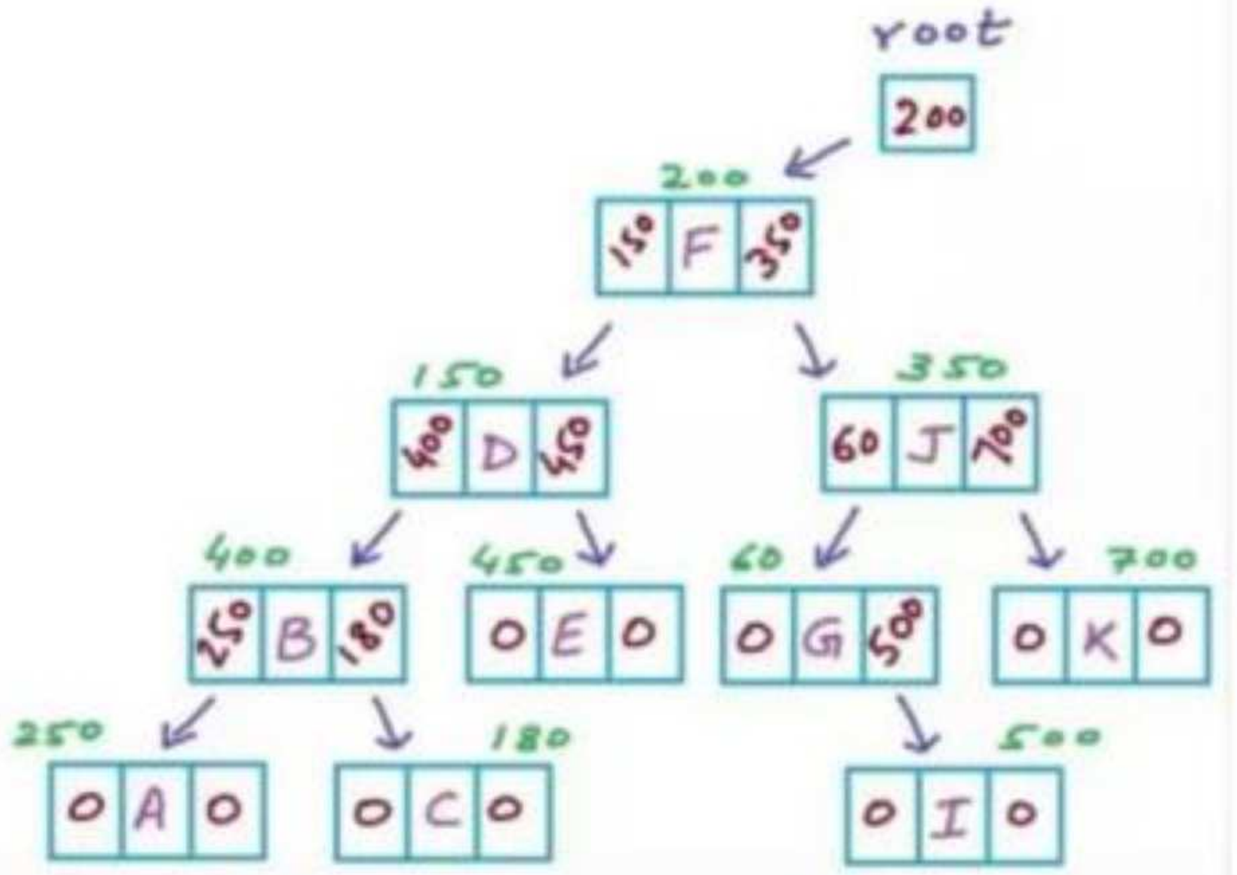


fig Binary tree

The preorder traversal output of the given tree is: A B D H I E C F G

Pre-order Pseudocode

```
struct Node{
    char data;
    Node *left;
    Node *right;
}
void Preorder(Node *root)
{
    if (root==NULL) return;
    printf ("%c", root->data);
    Preorder(root->left);
     Preorder(root->right);
 }
```



- The in-order traversal of a nonempty binary tree is defined as follows:
  - Traverse the left sub-tree in in-order
  - Visit the root node
  - Traverse the right sub-tree in inorder

- The in-order traversal outpu of the given tree is
  H D I B E A F C G

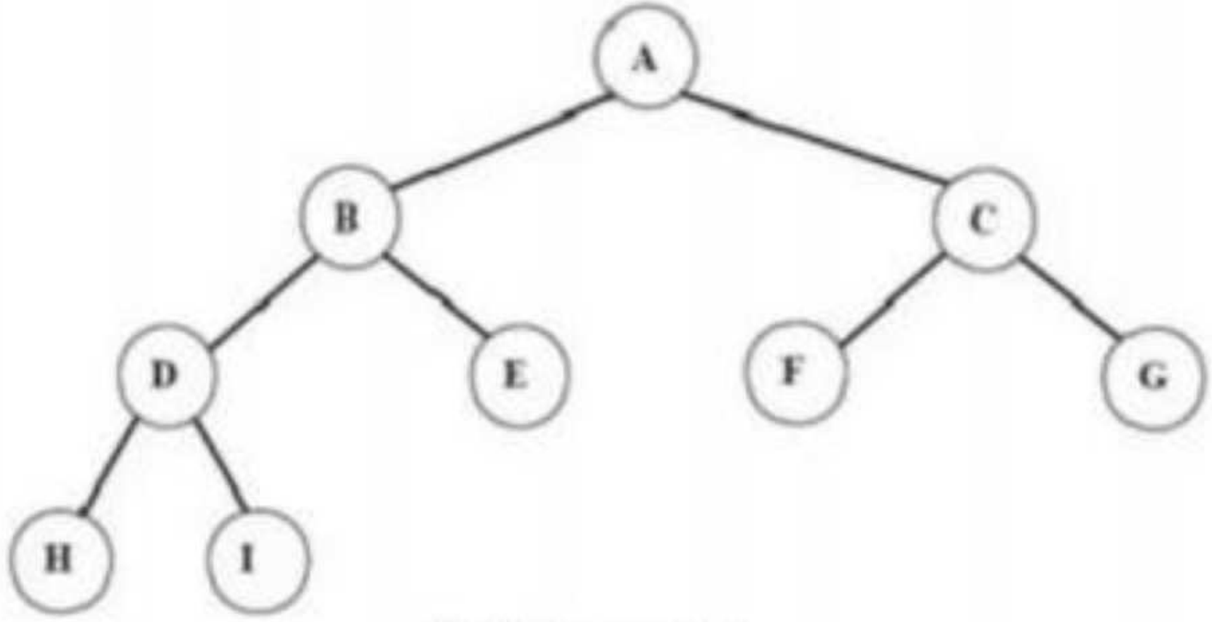


fig Binary tree

2 mins Summary

Topic

Tree. Queue, Recursion

Tree, Graph, Linked List

TEACHING WALLAH

THANK YOU